E02 · Especificación técnica

Frecuencias inteligentes SOBUSA: arquitectura, datos, API y decisiones técnicas

Roger Daza Matteo Del Sole Rafael Romero

Universidad del Norte · Reto Extreme Technologies – SOBUSA

Versión del 29 de octubre de 2026 · esquema `ai_frecuencias` en la revisión 0009

Índice

1. Alcance **1**

2. Arquitectura **2**

2.1. Principios . . . 2

2.2. Despliegue . . . 3

3. Datos **3**

3.1. Fuentes y decisión sobre los consolidados . . . 3

3.2. Calidad . . . 4

3.3. Lectura incremental . . . 4

3.4. Esquema propio . . . 4

4. Modelo de demanda **4**

4.1. Formulación y entrenamiento . . . 4

4.2. Incertidumbre y explicabilidad . . . 5

4.3. Ciclo de vida, monitoreo y aprendizaje continuo . . . 5

5. Optimización de frecuencia **5**

6. Alertas **7**

7. API REST **7**

8. Requisitos no funcionales **8**

9. Decisiones técnicas **9**

10. Riesgos y pendientes **10**

1. Alcance

El prototipo implementa los módulos obligatorios de la especificación v1.5 para tres rutas (2 SILENCIO, 16 CRA 54, 21 CL 72):

- lectura incremental y de solo lectura de la base del operador;
- validación de calidad;

- pronóstico de subidas a 30, 60 y 120 minutos con incertidumbre;
- recomendación de frecuencia y flota con función objetivo;
- alertas ALT-001 a ALT-010 con gestión y notificación;
- ciclo diario parametrizable con reentrenamiento controlado;
- monitoreo;
- API REST versionada y tablero web.

Quedan fuera el despliegue productivo definitivo y la modificación de tablas, triggers o scripts del operador (§3.2).

Este documento resume el diseño. El detalle operativo está en el repositorio: README, manual técnico, manual de despliegue, manual de usuario, `docs/openapi.json` y E07 (mapa de tablas y rendimiento).

2. Arquitectura

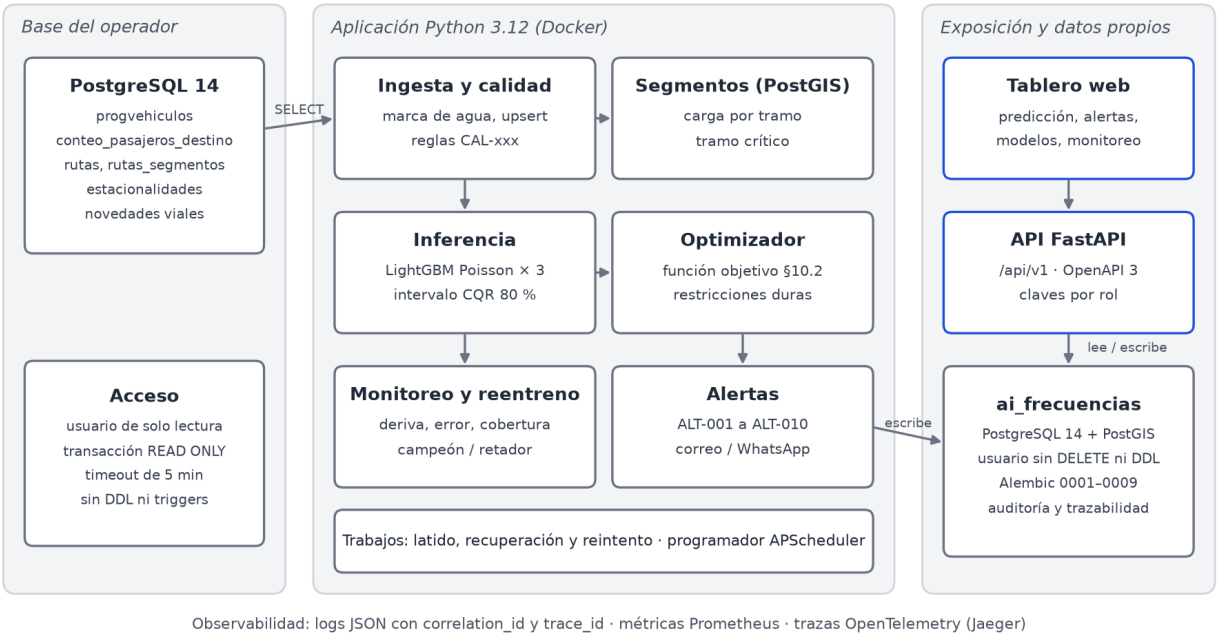


Figura 1: Componentes y flujo de datos. La base del operador solo se lee; todos los resultados se escriben en `ai_frecuencias`.

2.1. Principios

- **Separación de responsabilidades.** Cada paquete Python atiende una capa (Tabla 1). La API y el programador usan los mismos servicios de dominio.
- **No interferencia.** La base del operador se lee con un usuario de solo lectura, transacciones `READ ONLY` y `statement_timeout` de 5 minutos. Nunca se ejecuta DDL ni DML sobre ella.
- **Mínimo privilegio en la base propia.** La aplicación escribe con un usuario que no puede borrar, truncar ni alterar tablas. Solo las migraciones usan al dueño del esquema.
- **Procesos largos como trabajos.** La ingesta, la predicción por lotes y el reentrenamiento corren fuera de las peticiones HTTP, con estado, avance, latido, recuperación y reintento.
- **Configuración externa.** Todo parámetro de entorno va en variables. Todo umbral de negocio va versionado en PostgreSQL. No hay secretos en el repositorio.

- **Trazabilidad.** Cada predicción, recomendación y alerta apunta a la ejecución, el modelo, las variables y la regla que la produjeron.

Tabla 1: Componentes lógicos y tecnología.

Componente	Responsabilidad	Tecnología
API REST	Autenticación por rol, validación, consultas, comandos, OpenAPI 3	FastAPI, Pydantic
Acceso a la fuente	Consultas incrementales parametrizadas, solo lectura	SQLAlchemy 2.1, psycopg2
Calidad y variables	Reglas CAL-xxx, serie de 15 min, 22 variables sin fuga	pandas, NumPy
Inferencia	Carga verificada por SHA-256, cuantiles CQR, explicación local	LightGBM 4.7
Optimización	Función objetivo con restricciones duras (§10.2)	Python (búsqueda entera exacta)
Alertas	Reglas, deduplicación, estados, notificación	Motor propio, SMTP, WhatsApp Cloud API
Programación	Ciclo por cron, reintentos, reanudación	APScheduler
Persistencia	Esquema `ai_frecuencias`, migraciones, rol de aplicación	PostgreSQL 14, PostGIS 3.5, Alembic
Observabilidad	Logs JSON, métricas, trazas	structlog, Prometheus, OpenTelemetry
Calidad del código	Pruebas, cobertura, análisis estático	pytest, unittest.mock, SonarQube

2.2. Despliegue

Docker Compose define:

- `db`: PostgreSQL 14 con PostGIS.
- `migraciones`: una sola corrida con el dueño del esquema; aplica Alembic y crea el usuario restringido.
- `api`: arranca solo si las migraciones terminaron bien, con el usuario restringido.
- Perfiles opcionales: `programador`, `observabilidad` (Jaeger) y `calidad` (SonarQube).

La imagen corre como usuario sin privilegios. Los modelos se montan desde `modelos/`. La integración continua (GitHub Actions) tiene tres trabajos:

- **Pruebas:** aplica, revierte y reaplica las migraciones y corre las pruebas.
- **Calidad:** levanta su propio servidor SonarQube y falla con hallazgos críticos o bloqueantes.
- **Imagen:** construye la imagen Docker.

3. Datos

3.1. Fuentes y decisión sobre los consolidados

La demanda se construye a partir de `conteo_pasajeros_destino`: 176 millones de lecturas de sensores entre 2023 y 2025. Se une con `progvehiculos` por `idprogramacion` y se agrega por ruta y franja de 15 minutos. Los consolidados `timbradas_*` que propone la especificación no se usaron como fuente del modelo, por dos razones:

- `timbradas_segmentos` solo cubre ocho semanas de 2023.
- Sus horas están desplazadas dos horas respecto de los sensores: la correlación franja a franja sube de 0,47 a 0,71 al corregir el desfase.

La suma de lecturas por viaje tiene correlación 0,9999 con `numeropasajeros` de la programación.

Los scripts de segmentación del operador escriben en la base, así que no se pueden ejecutar con acceso de solo lectura. Su lógica se replicó con PostGIS: asignación de cada lectura al segmento más cercano con índice GiST y carga balanceada. Coincide en 99,7 % con `timbradas_segmentos`.

3.2. Calidad

Las reglas de calidad se derivan del análisis exploratorio. Cubren:

- días sin lecturas, que se tratan como faltantes y no como cero;
- lecturas imposibles y saltos del contador acumulado;
- sensor de bajadas poco fiable;
- viajes con fin anterior al inicio;
- relaciones faltantes;
- distancias de lectura a la ruta mayores a 300 m.

Cada problema queda en `data_quality_issue` con regla, dimensión (completitud, validez, consistencia, oportunidad, relacional), fila y causa. Se consulta por lote en `GET /api/v1/data-quality`.

3.3. Lectura incremental

El ciclo lee desde la última franja del histórico: `max(demand_history.ts)` + 15 min. La carga es un upsert por clave natural, así que repetir un lote no duplica: se probó con 808.345 registros. Cada lote queda en `execution_log` con su ventana. Si un ciclo cae después de ingerir, el siguiente procesa desde la marca de agua del último ciclo exitoso, con un tope de 7 días; así no se pierden predicciones ni alertas. El mapa completo de tablas, consultas y planes de ejecución está en E07.

3.4. Esquema propio

El esquema `ai_frecuencias` tiene 17 tablas, todas creadas con Alembic, en `TIMESTAMPTZ` y con claves naturales protegidas por `UNIQUE`:

- histórico analítico (`demand_history`, `trip_history`);
- resultados (`prediction`, `feature_snapshot`, `recommendation`, `alert`, `alert_history`, `notification`);
- gobierno (`model_registry`, `operational_parameter`, `api_key`);
- operación (`execution_log`, `data_quality_issue`, `monitoring_metric`);
- segmentación (`route_segment` con geometría PostGIS, `segment_observation`, `segment_profile`).

4. Modelo de demanda

4.1. Formulación y entrenamiento

Pronóstico supervisado de subidas por ruta y franja, con un modelo LightGBM de objetivo Poisson por horizonte (30, 60 y 120 min):

- **Variables.** 22 variables que solo usan información anterior al instante de predecir: rezagos del mismo día, día anterior, semana y año; promedio de 4 semanas; minuto del día; tipo de día y festivos; estacionalidades; viajes habituales e impacto de novedades. Una prueba automática altera el futuro y verifica que la predicción no cambie.
- **División cronológica.** Entrenamiento, validación y prueba reservada, de mayo a noviembre de 2025.
- **Ajuste.** Hiperparámetros con Optuna (15 intentos, validación temporal de 4 pliegues, semilla 42).
- **Registro.** Datos, semilla, hiperparámetros, versión de variables, métricas, artefacto y su SHA-256 quedan en `model_registry`.

Tabla 2: Error en el periodo de prueba (mayo a noviembre de 2025, 44.626 franjas).

Horizonte	WAPE modelo	WAPE base 4 sem.	WAPE sem. pasada	MAE modelo / base
30 min	10,1 %	14,9 %	18,4 %	16,9 / 24,9
60 min	10,5 %	14,9 %	18,4 %	17,6 / 24,9
120 min	10,8 %	14,9 %	18,4 %	18,2 / 24,9

El reporte por ruta, tipo de día y hora (pico y valle) y el sesgo están en `modelos/reporte_entrenamiento.json` y en el artículo técnico (`paper/main.pdf`). En días especiales el error es de 14–16 %, frente a 99 % de la línea base.

4.2. Incertidumbre y explicabilidad

Los cuantiles p10 y p90 se calibran con predicción conforme (CQR). El intervalo del 80 % cubre 81,6–82,5 % en prueba. Fuera del servicio habitual el intervalo se marca como no calibrado.

Cada predicción guarda las variables usadas (`feature_snapshot`) y una explicación local: el efecto porcentual de las variables más influyentes, por ejemplo "hora del día +38 %". La importancia global (ganancia y SHAP) está en el reporte de entrenamiento.

4.3. Ciclo de vida, monitoreo y aprendizaje continuo

Carga verificada. El modelo se carga una vez y se verifica su huella; un paquete alterado se rechaza.

Monitoreo (§9.4). En cada ciclo se calculan y guardan en `monitoring_metric`:

- deriva de datos: PSI de las subidas recientes frente a las 8 semanas previas al corte del modelo;
- error móvil (WAPE de 7 días);
- cobertura del intervalo;
- volumen de predicciones;
- porcentaje de alertas descartadas.

Último ciclo: WAPE de 11–13 %, cobertura de 76–81 % y PSI de 0,08–0,21.

Reentrenamiento (§9.5, RF-018, RF-019):

- Se dispara por degradación (ALT-010 por error o cobertura), por deriva (PSI $> 0{,}25$) o por periodicidad (7 días), con al menos 24 horas entre intentos.
- El candidato se entrena con los datos anteriores a una ventana de comparación que ninguno de los dos modelos vio.
- Solo se activa si mejora el WAPE o la desviación de Poisson sin empeorar la otra y supera a la línea base.
- El modelo anterior queda registrado para revertir con `POST /models/{id}/activate`.
- Los paquetes reemplazados se archivan, no se borran.

5. Optimización de frecuencia

Para cada ruta y hora se eligen los buses por hora b que minimizan la función objetivo de referencia (§10.2):

$$\min_{b\in\mathbb{Z}} \alpha\,\frac{b\,T_c}{60} \;+\; \beta\,\frac{D}{2b} \;+\; \gamma\,\mathbb{E}\big[(L - b\,C\,o^{*})^{+}\big] \;+\; \delta\,|b - b_0|$$
$$\text{s.a. } b \geq \lceil 60/H_{\text{máx}} \rceil, \qquad b\,C\,o_c \geq \mathbb{E}[L], \qquad b \leq \lfloor 60/H_{\text{mín}} \rfloor, \qquad V(b) \leq F,$$

donde:

- T_c es la duración del ciclo de la ruta para el tipo de día y la franja;
- D es la demanda prevista de la hora;
- $L = D \cdot f$ es la carga en el tramo más cargado (f es el factor de carga medido por franja);
- C es la capacidad;
- o^* y o_c son la ocupación objetivo (80 %) y la crítica (90 %);
- b_0 es la oferta actual, observada o habitual;
- $V(b)$ son los vehículos requeridos;
- F es la flota disponible.

Los cuatro términos están en unidades equivalentes a una hora de operación de un vehículo:

- costo de operación;
- horas de espera de los pasajeros, con espera media igual a la mitad del headway;
- pasajeros esperados por encima de la ocupación objetivo;
- cambio frente a lo que se opera.

Incertidumbre. L se modela como normal con media $\mathbb{E}[L]$ y desviación $(p_{90} - p_{50})\, f/1{,}2816$, tomadas del intervalo del modelo. El exceso esperado tiene forma cerrada $\sigma\varphi(z) + (\mu - c)(1 - \Phi(z))$.

Solución. El dominio factible tiene a lo sumo 30 enteros, así que la búsqueda exhaustiva da el óptimo global de forma exacta y auditable. Para cada recomendación se guardan los componentes del costo, las tres mejores alternativas, la restricción que limita la decisión y la sensibilidad a cada peso multiplicado por 0,5 y por 2. Si las restricciones no se pueden cumplir, la recomendación se marca *no viable* con su motivo, en lugar de forzar una cifra.

Fin del servicio. Para cada ruta y día se calcula la ventana de despachos:

- **Apertura:** la menor entre la primera salida programada del día y la mediana de las 4 semanas previas del mismo día de la semana.
- **Cierre:** la mayor entre la última salida programada del día y esa misma mediana.

Las horas con menos de 25 % dentro de la ventana no se recomiendan, y en la hora parcial los buses operados se comparan como tasa dentro del servicio. Así, después de la última salida el sistema no pide buses por el mínimo de servicio.

Pesos y línea base. Los pesos de referencia son $\alpha = 1$, $\beta = 0{,}1$, $\gamma = 1$ y $\delta = 0{,}5$. Están versionados en `operational_parameter` y requieren validación de SOBUSA. Con estos valores, recalculando las 748 horas-ruta del 1 al 14 de diciembre de 2025:

- El óptimo coincide con la línea base de §10.3 ($\lceil$carga p90 / capacidad efectiva$\rceil$ con mínimo de servicio) en el 73 % de los casos y recomienda menos buses en el 27 %.
- Frente a lo operado, propone 8,4 buses por hora en promedio frente a 8,5.
- La flota se redistribuye: SILENCIO pasa de 7,7 a 6,6 buses por hora y CL 72 de 4,6 a 5,5.
- La mediana de ocupación esperada es 65 %.

6. Alertas

Tabla 3: Catálogo implementado. Todos los umbrales se versionan en `operational_parameter`.

Código	Severidad	Condición
ALT-001	preventiva / crítica	Carga p90 con la oferta actual $\geq$ umbral de capacidad; crítica si la recomendación no es viable
ALT-002	crítica	Ocupación esperada con la oferta actual $\geq$ 90 %
ALT-003	preventiva	Oferta actual menor que la recomendada, más allá de la tolerancia (2 buses/h y 15 %)
ALT-004	informativa	Ocupación esperada $<$ 30 % con más buses de los recomendados
ALT-005 / 007	preventiva	Demanda real fuera del intervalo con desvío $\geq$ 15 %
ALT-006	crítica	En una hora, al menos 2 despachos programados y al menos 25 % salieron cancelados, sin iniciar o fuera de plazo (`excedeplazosalida`)
ALT-008	preventiva	Consolidado con valores atípicos
ALT-009	crítica	Día sin lecturas, datos atrasados más de 2 periodos o fuente no disponible
ALT-010	crítica	WAPE móvil $>$ tolerancia o cobertura del intervalo $<$ 65 %, con al menos 3 días observados

Cada alerta guarda empresa, ruta, ventana, horizonte, valor observado, esperado y umbral, puntuación de anomalía, explicación, variables influyentes, recomendación y referencias a la predicción, la recomendación, el modelo y la ejecución.

- **Deduplicación.** Se hace por clave mientras la alerta está abierta.
- **Estados.** Abierta, reconocida, asignada, cerrada y descartada, con bitácora.
- **Notificación.** Las nuevas críticas se notifican por correo y WhatsApp, con timeout, reintento y circuit breaker.

ALT-006 da unas 4 alertas diarias entre las tres rutas con los datos de diciembre de 2025.

Para no saturar al operador, el tablero agrupa las alertas por ruta, código y día (en la demostración, 775 pendientes caben en 178 grupos) y permite gestionarlas en lote. El tablero incluye un mapa de Barranquilla (Leaflet y OpenStreetMap) con los segmentos coloreados por ocupación p90 y el tramo crítico de cada ruta. La pantalla inicial *Hoy* traduce la recomendación vigente de cada ruta a una acción en lenguaje operativo (por ejemplo, “Agregar 2 buses entre 06:15 y 07:15: de 10 a 12 por hora”) sin recalcular, a partir de lo que el ciclo ya guardó.

7. API REST

Convenciones.

- Ruta base `/api/v1`, JSON UTF-8 y fechas ISO-8601 con zona `America/Bogota`.
- Validación con Pydantic.
- Errores uniformes: `{error:{codigo, mensaje, detalle, correlation_id}}`.
- Paginación (`page`, `size`) y filtros en las colecciones.
- Cabecera `X-Correlation-Id` y autenticación por `X-API-Key` con roles lector, operador y administrador.
- Swagger UI restringido por ambiente.

El contrato completo está en `docs/openapi.json` (OpenAPI 3.1, 29 rutas).

Tabla 4: Endpoints.

Método y ruta	Rol	Finalidad
`POST /frequency/executions`	operador	Lanzar el ciclo (202, trabajo asíncrono)
`GET /frequency/executions[/{id}]`	lector	Estado, etapa, avance, intento, ventana, métricas, errores
`POST /frequency/executions/{id}/retry`	operador	Reintentar un ciclo fallido o interrumpido
`POST /predictions/run`	operador	Inferencia para ruta y referencia
`GET /predictions[/{id}/segments]`	lector	Pronósticos por filtros; desagregación por segmento
`POST /recommendations/calculate`	operador	Frecuencia y flota optimizadas
`GET /recommendations`	lector	Recomendaciones por filtros
`GET /overview`	lector	Pantalla *Hoy*: acción por ruta para la hora actual y la siguiente, franja del día y alertas
`GET /alerts[/summary|/{id}]`	lector	Alertas por estado, severidad, código, ruta, fecha y ventana
`GET /alerts/groups`	lector	Alertas agrupadas por ruta, código, severidad y día
`POST /alerts/status`	operador	Cambio de estado en lote (hasta 500), con bitácora por alerta
`PATCH /alerts/{id}/status`	operador	Reconocer, asignar, comentar, cerrar o descartar
`GET /models` · `POST /models/{id}/activate`	lector · admin.	Registro de modelos; activación auditada
`GET /monitoring[/history]`	lector	Métricas de monitoreo vigentes e histórico
`GET /parameters` · `PUT /parameters/{clave}`	lector · admin.	Umbrales y pesos versionados
`GET /segments[/profile]`	lector	Catálogo y perfil de carga por segmento
`GET /segments/map`	lector	GeoJSON del mapa: segmentos con carga y ocupación p90, tramo crítico por ruta
`GET /data-quality` · `GET /notifications`	lector	Problemas de calidad por lote; envíos
`GET /export/{recurso}`	lector	CSV o XLSX filtrado
`GET /salud` · `GET /metrics`	público	Salud y métricas Prometheus

8. Requisitos no funcionales

Tabla 5: Cumplimiento y evidencia.

Requisito	Cómo se cumple	Evidencia
RNF-001 Disponibilidad	Latido, recuperación de ejecuciones sin latido, reanudación automática y ventana pendiente desde la última marca exitosa	Ciclo real fallido y reintentado; API matada con `docker kill` a mitad de un ciclo y reanudada sola (`caida_api.json`)
RNF-002 Rendimiento	Consultas paginadas indexadas; predicciones precalculadas por el ciclo	p95 de 146 ms con 10 usuarios (`carga_api.json`)
RNF-003 Ingesta	Upsert por lotes con `COPY` a tabla temporal	808.345 registros en 149 s, pico de 964 MB
RNF-004 Asincronía	Ciclo como trabajo con etapa, avance y reintento (API y programador); exclusión entre procesos por índice único	`humo_docker.json`: 202 y 409
RNF-005 Escalabilidad	API sin estado de sesión; procesamiento por ruta y fecha	—
RNF-006 Observabilidad	Logs JSON con `correlation_id` y `trace_id`, Prometheus, trazas OpenTelemetry de HTTP, SQL y etapas	`tests/test_trazas.py`

Requisito	Cómo se cumple	Evidencia
RNF-007 Mantenibilidad	Arquitectura modular, 322 pruebas, 79 % de cobertura	`reporte-pruebas.xml`
RNF-008 Portabilidad	Docker Compose con base, migraciones, API y perfiles	`humo_docker.json`
RNF-009 Compatibilidad	`/api/v1`, OpenAPI 3.1, 9 migraciones reversibles probadas en CI	`docs/openapi.json`
RNF-010 Resiliencia	Timeout, reintento con espera creciente y circuit breaker en fuente y notificaciones	`tests/test_resiliencia.py`
RNF-011 Trazabilidad IA	Ejecución, modelo, variables, regla y parámetros enlazados en cada salida	Esquema E07
RNF-012 Accesibilidad	Navegación por teclado, contraste $\geq$ 5,7:1, modo oscuro y móvil	`e2e_tablero.json`
RNF-013 Calidad de código	SonarQube: 0 bugs, 0 vulnerabilidades, 0 code smells, 0 puntos de seguridad, 0,3 % de duplicación	`sonarqube.json`
RNF-014 Pruebas en CI	GitHub Actions: pruebas con PostGIS y análisis con SonarQube propio	`.github/workflows/ci.yml`

9. Decisiones técnicas

Tabla 6: Decisiones, alternativas consideradas y razones.

Decisión	Alternativas	Razón
Sensores crudos como fuente de demanda	Consolidados `timbradas_*`	Los consolidados entregados cubren ocho semanas y tienen horas desplazadas; los sensores cubren tres años
LightGBM Poisson por horizonte	XGBoost, Random Forest, redes temporales	Cinco variantes de boosting difieren en décimas de punto; LightGBM es el más rápido. La mejora viene de los datos, no de la familia
Intervalos CQR	Cuantiles sin calibrar, ancho constante	Mejor cobertura (82 %) con ancho similar
Búsqueda exacta para la frecuencia	OR-Tools, SciPy MILP, PuLP	Dominio de a lo sumo 30 enteros por hora: óptimo global exacto, sin dependencias y auditable. Si se agrega flota compartida entre rutas, el mismo modelo pasa a MILP
ALT-006 por hora con despachos fallidos	Huecos entre salidas	`fechasalida` es la hora programada; contar cancelados, sin iniciar y fuera de plazo (marca del operador) mide la falla real y da unas 4 alertas al día
Índice único parcial para un ciclo a la vez	Candado en memoria, `pg_advisory_lock`	Funciona entre procesos y máquinas, sobrevive a reinicios y es visible en la tabla
Latido por instancia	Solo tiempo de inicio	Distingue un ciclo largo (reentrenamiento) de uno abandonado
Usuario de aplicación sin DELETE ni DDL	Usuario dueño	Mínimo privilegio; la historia no se puede borrar desde la aplicación
Verificador SCRAM generado en cliente	`ALTER ROLE ... PASSWORD` en claro	La clave no viaja ni queda en logs del servidor
Modelos fuera de git	Versionarlos en el repositorio	Son artefactos con huella registrada; git alteraba sus fines de línea
SonarQube como servicio del job de CI	Servidor externo con secretos	Reproducible y sin credenciales externas
Instrumentación SQLAlchemy con `skip_dep_check`	Fijar SQLAlchemy 2.0	La instrumentación declara soporte hasta 2.0; con 2.1 funciona y está cubierta por pruebas

10. Riesgos y pendientes

- **Parámetros operativos sin validar.** Capacidad (50 pasajeros), headway máximo (10 min de día) y pesos de la función objetivo. En 7–20 % de los viajes la carga medida supera 50 pasajeros. Mitigación: son parámetros versionados en la base y se ajustan sin desarrollo.
- **Fuente.** Si el operador corrige lecturas pasadas, el ciclo no las relee. Mitigación: recarga idempotente del histórico. Pendiente: recarga por rango.
- **Notificaciones simuladas** hasta recibir credenciales de SMTP y WhatsApp.
- **Una instancia de API por host,** supuesto de la recuperación al arrancar. Para varias réplicas, la recuperación queda solo por latido.
- **Video de demostración (E11) y plan de evolución (E12)** a cargo del equipo.